

CSE4067

Introduction to Blockchain Programming

Goshgar Can Ismayilov

goshgar.ismayilov@boun.edu.tr

<https://www.linkedin.com/in/gismayilov/>

05.11.2024

Smart Contracts

It is **possible** to **apply multiple modifiers to a function**. The modifiers will be resolved sequentially, starting from left to right. In the example below, a user can only call the **kill** function if he/she is the owner of the contract and has an account balance greater than 1337 ETH.

```
contract owned {  
    address public owner;  
  
    constructor() public {  
        owner = msg.sender;  
    }  
  
    modifier onlyOwner {  
        require(msg.sender == owner);  
        _; // Actual function code is injected here  
    }  
  
    modifier isRich {  
        require(msg.sender.balance > 1337 ether);  
        _; // Actual function code is injected here  
    }  
  
    function kill() public onlyOwner isRich {  
        selfdestruct(owner);  
    }  
}
```

Smart Contracts

Solidity allows to overload functions, i.e., to define the same function twice with different signatures. This can be helpful if a function needs to be adapted to certain situations.

Example

```
function sendEther(uint amount) {  
    require(this.balance >= amount);  
    payable(msg.sender).transfer(amount);  
}
```

```
function sendEther(uint amount, address payable to) {  
    require(this.balance >= amount);  
    to.transfer(amount);  
}
```

If `sendEther()` is called without the address argument, the Ether will be sent to the caller. Otherwise, it will be sent to the address passed as a parameter to the function.

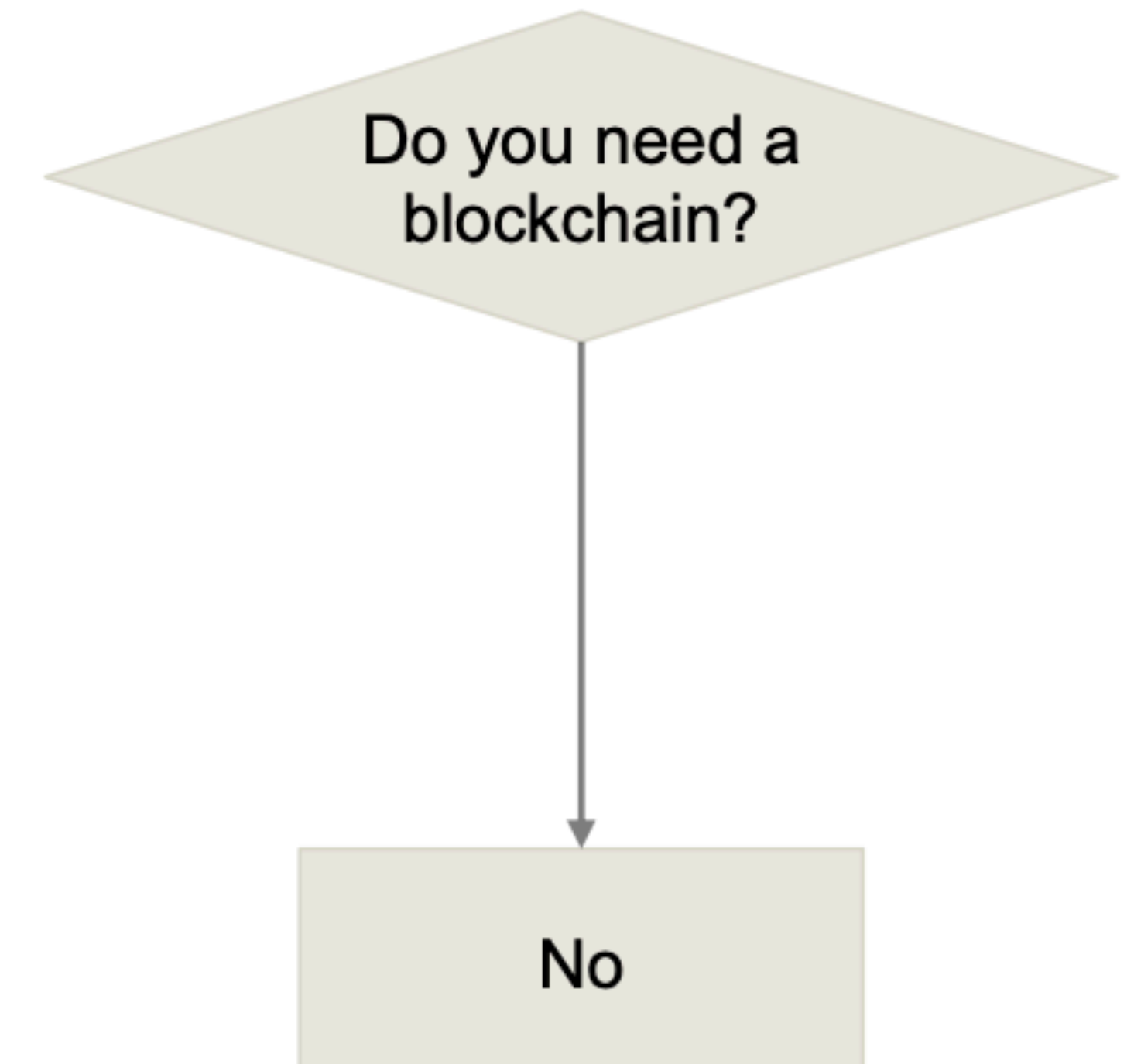
Smart Contracts

The hype of blockchain has led to a large number of software applications using blockchain technology. Often used by startups or driven by innovation departments in enterprises.

However, the real-world adoption of contracts still is negligible. Currently, the largest dApp¹ using an Ethereum smart contract has less than ~11000 transactions per day (<https://dappradar.com/rankings/protocol/ethereum>).

Finding a use case

- Understand the problem domain and the blockchain technology well
 - Blockchain might be a potential solution if:
 - Multiple parties are involved **and**
 - The parties do not trust each other or have different interests **and**
 - Shared write access is required **and**
 - All writes to the database need to be (publicly) verifiable



Smart Contracts

As in traditional software engineering, the first step for creating an application is to model the business process.

Identification

- Identify the involved parties, systems, and their relationship.
- Identify the necessary interactions between the parties and systems.
- Identify the information that is exchanged between the parties and systems.
- Identify the system boundaries.
- Diagrams can help to get the big picture.

Modelling

- Derive concrete models from the identified parties and systems.
- Define concrete messages that are exchanged between the systems and parties.
- Define a concrete data model used by the system.
- Derive concrete interfaces for interaction with the systems and parties.
- The overall architecture is usually modeled at a high abstraction level using architecture diagrams.
- The concrete software is modeled at a lower abstraction level using class diagrams.

Example: Blood Donation

The DRK (“Deutsches Rotes Kreuz”) wants to digitize the blood donation process and make it more transparent. Therefore, it analyzes different technological solutions. As a first step, the DRK wants to track the supply chain from the blood extraction to the transfusion. The overall process is listed below.

Current simplified process:

- Person goes to the DRK and donates blood
- Blood is analyzed in laboratory and labeled
- Blood product is sent to hospital
- Hospital checks if a patients blood is compatible with the product
- Blood is transfused

Involved Parties:

- DRK – Extracts the blood from the donor
- Laboratory – Analyzes the blood and creates blood products
- Donor – Donates his/her blood
- Hospital – Transfuses the blood
- Patient – Receives the blood transfusion

Involved Systems:

- Laboratory management system (LMS) in the laboratory that analyzes the blood.
- The hospital information system (HIS) used in the hospital to manage patients and processes.

Interactions:

- DRK extracts blood from Donor
- DRK sends blood to laboratory
- Laboratory analyzes blood
- Hospital sends request to laboratory
- Laboratory sends blood to hospital
- Hospital transfuses blood to patient

Example: Blood Donation

Assessing blockchain and contracts as potential solution architecture.

Multiple parties are involved ✓

- At least four parties are involved

The parties do not trust each other or have different interests –

- Questionable, some patients might not trust the DRK and therefore do not donate.

Shared write access is required ✓

- DRK needs to read and write
- The laboratory needs to read and write
- The hospital needs to read and write

All writes to the database need to be publicly verifiable ✓

- The donor needs to be able to track what happened with his/her donation.
- In case of an accident, the blockchain could help to identify the root cause and the responsible party.

Example: Blood Donation

In the Ethereum ecosystem two kinds of entities exist, externally owned accounts (EOAs) and contracts. Transactions are always issued by EOAs and usually controlled by an individual or a party. If an entity needs to be interactive and provide some on-chain functionality it is a candidate for a contract.

EOAs

Donor – Person who donates blood identified by their unique wallet address

DRK – Institution that runs blood donation events

Laboratory – Laboratory that takes the blood donation and creates blood products out of it

Hospital – The institution that transfuses the blood of the donor to a patient

Patient – The patient who gets a blood transfusion

Contracts

Blood Donation – Contract owned by the DRK, it tracks the time and date of the donation. Additionally, it records whenever the donation is passed to another party, e.g., from the DRK to the laboratory.

Example: Blood Donation

In the Ethereum ecosystem transactions and messages are the only way for entities to interact.

Interactions

DRK → CONTRACT_CREATION:

Creates a blood donation (BD) and deploys it on the blockchain. The address of the donor is passed via constructor and unchangeable.

DRK → BD.[sendToLaboratory](#)([address](#) laboratory):

The DRK issues a transaction to the BD when the blood donation is sent to the laboratory. The state variable for the laboratory is set.

Laboratory → BD.[sendToHospital](#)([address](#) hospital):

The Laboratory issues a transaction to the BD when the blood donation is sent to the hospital. The state variable for the hospital is set. Only the hospital can do the transfusion.

Hospital → BD.[transfuse](#)([address](#) patient):

The hospital issues a transaction to the BD when the blood is transfused to a patient. A transfusion can only happen once.

Example: Blood Donation

DRK: 0x91A0639dDe409c126f058e33D743b1253738C8b9

PK: 0x3aae751e36ddffd4f7d5ff4bee409583a54df823111a30f780c18cd73ebb02f8

Laboratory: 0x3aDDBa6E0C56EE1357Bb9796b20480880cA37E81

PK: 0xdbbe7d4d5460f6a6e086579a0acf071b652b6ed5ae0374d704a949cbb0b740a65

Hospital: 0x582FFFacdBFDaF1936672886035ea561FF669a44

PK: 0x8ae80121c7bc29a51eb4401754928051063a8dace9d35496dd26d0c4a1a0640c

Patient: 0x0780aFf9177d78E86Fc03158D504652f88c4D1bc

PK: 0x0a18f4e53a62e97b613ed94d0f411de327e0d1a0d5533c685042cae420aacbfcb

Donor: 0x39bc67dBb1f5203AF048699233b29Dec903389A4

PK: 0x5aed62bff0a98533345482fb91ac80388869e6bdd5ad53c19b54a37468a5cb2d

Example: Blood Donation

// File: BloodDonation.sol

pragma solidity >=0.4.22 <0.6.0;

```
contract BloodDonation {
    address donor;
    address drk;
    address laboratory;
    address hospital;
    address patient;
    bool isTransfused = false;
    modifier onlyDRK() {
        require(msg.sender == drk);_
    }
    modifier onlyLaboratory() {
        require(msg.sender == laboratory);_
    }
    modifier canTransfuse() {
        require(!isTransfused);
        require(msg.sender == hospital);
        _
    }
}
```

// ... continues here

```
constructor(address _donor) public {
    drk = msg.sender;
    donor = _donor;
}
function sendToLaboratory(address _laboratory) onlyDRK {
    laboratory = _laboratory;
}
function sendToHospital(address _hospital) onlyLaboratory {
    hospital = _hospital;
}
function transfuse(address _patient) canTransfuse {
    patient = _patient;
    isTransfused = true;
}
}
```

Ethereum Token Standards

Tokens are smart contracts that implement a standardized interface. They are frequently used by many decentralized applications running on the Ethereum ecosystem.

Depending on the actual use case, different token standards exist. The most common use cases and standards are:

- Decentralized Finance, ICOs, crowd funding (mostly ERC 20)
- Gaming (mostly ERC 20 and ERC 721)

Token classes:

1. Fungible tokens: All tokens are indistinguishable and have exactly the same value.



2. Non-fungible tokens (NFT): Each token is uniquely identifiable by an ID.



3. ERC-1155 Multi-token standard: Allows for each token ID to represent a new configurable token type, which may have its own metadata, supply and other attributes.



ERC-20 Token Standard

The **ERC20** standard was first proposed by Fabian Vogelsteller and Vitalik Buterin in November 2015. The specification **defines an interface** that a contract must implement to be ERC20 compliant. It does **not specify** the **actual implementation** of the functions.

It is the most commonly used token standard in the Ethereum network with more than 450,000 contracts deployed on the main network. Each **ERC20 contract** defines a class of **fungible tokens**.

Interface

```
interface IERC20 {  
    function totalSupply() public view returns (uint);  
    function balanceOf(address tokenOwner) public view returns (uint balance);  
    function allowance(address tokenOwner, address spender) public view returns (uint remaining);  
    function transfer(address to, uint tokens) public returns (bool success);  
    function approve(address spender, uint tokens) public returns (bool success);  
    function transferFrom(address from, address to, uint tokens) public returns (bool success);  
  
    event Transfer(address indexed from, address indexed to, uint tokens);  
    event Approval(address indexed tokenOwner, address indexed spender, uint tokens);  
}
```

ERC-20 Token Standard

```
contract ERC20 is Context, IERC20, IERC20Metadata {  
  
    mapping (address => uint256) private _balances;  
    mapping (address => mapping (address => uint256)) private _allowances;  
    uint256 private _totalSupply;  
  
    string private _name;  
    string private _symbol;  
  
    constructor (string memory name_, string memory symbol_) {  
        _name = name_;  
        _symbol = symbol_;  
    }  
}
```

ERC-20 Token Standard

```
function name() public view virtual override returns (string memory){  
    return _name;  
}
```

Name: The function returns the token's name.

ERC-20 Token Standard

```
function symbol() public view virtual override returns (string memory){  
    return _symbol;  
}
```

Symbol: This function returns the token's emblem (typically a few letters or characters representing the token).

ERC-20 Token Standard

```
function decimals() public view virtual override returns (uint8){  
    return 18;  
}
```

Decimals: This function returns the number of decimal digits that the token can be divided into.

ERC-20 Token Standard

```
function totalSupply() public view virtual override returns (uint256) {  
    return _totalSupply;  
}
```

```
function balanceOf(address account) public view virtual override returns (uint256) {  
    return _balances[account];  
}
```

TotalSupply: This function returns the entire number of tokens that have been generated for a specific project.

BalanceOf: This function returns the token balance held by a specified address.

ERC-20 Token Standard

```
/**
 * @dev See `IERC20.transfer`.
 *
 * Requirements:
 *
 * - `recipient` cannot be the zero address.
 * - the caller must have a balance of at least `amount`.
 */
function transfer(address to, uint256 amount) public virtual override returns (bool) {
    address owner = _msgSender();
    _transfer(owner, to, amount);
    return true;
}
```

Transfer: This function enables an address to transmit tokens to another address.

ERC-20 Token Standard

```
/**
 * @dev Moves tokens `amount` from `sender` to `recipient`.
 *
 * Emits a `Transfer` event.
 *
 * Requirements:
 *
 * - `from` cannot be the zero address.
 * - `to` cannot be the zero address.
 * - `from` must have a balance of at least `amount`.
 */
function _transfer(address from, address to, uint256 amount) internal virtual {
    require (from != address(0), "ERC20: transfer from the zero address");
    require (to != address(0), "ERC20: transfer to the zero address");

    _beforeTokenTransfer (from, to, amount);

    uint256 fromBalance = _balances[from];
    require (fromBalance >= amount, "ERC20: transfer amount exceeds balance");
    unchecked{
        _balances[from] = fromBalance - amount ;
    }
    _balances[to] += amount;
    emit Transfer (from, to, amount);
    _afterTokenTransfer (from, to, amount);
}
```

ERC-20 Token Standard

```
/**
 * @dev See `IERC20.approve`.
 *
 * Requirements:
 *
 * - `spender` cannot be the zero address.
 */
function approve(address spender, uint256 amount) public virtual override returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, amount);
    return true;
}
```

Approve: With this function, an address can authorize another address to spend tokens on its behalf.

ERC-20 Token Standard

```
/**
 * @dev Sets `amount` as the allowance of `spender` over the `owner`'s tokens.
 *
 * This is internal function is equivalent to `approve`, and can be used to
 * e.g. set automatic allowances for certain subsystems, etc.
 *
 * Emits an `Approval` event.
 *
 * Requirements:
 *
 * - `owner` cannot be the zero address.
 * - `spender` cannot be the zero address.
 */
function _approve(address owner, address spender, uint256 amount) internal virtual {
    require(owner != address(0), "ERC20: approve from the zero address");
    require(spender != address(0), "ERC20: approve to the zero address");

    _allowances[owner][spender] = amount;
    emit Approval(owner, spender, amount);
}
```

ERC-20 Token Standard

```
/**
 * @dev See `IERC20.allowance`.
 */
function allowance(address owner, address spender) public view virtual override returns (uint256) {
    return _allowances[owner][spender];
}
```

Allowance: This function returns the number of tokens an authorized address is permitted to spend on behalf of another address.

ERC-20 Token Standard

```
/**
 * @dev See `IERC20.transferFrom`.
 *
 * Emits an `Approval` event indicating the updated allowance. This is not
 * required by the EIP. See the note at the beginning of `ERC20`;
 *
 * Requirements:
 * - `from` and `to` cannot be the zero address.
 * - `from` must have a balance of at least `amount`.
 * - the caller must have allowance for `from`'s tokens of at least
 *   `amount`.
 */
function transferFrom(address from, address to, uint256 amount) public virtual override returns (bool) {
    address spender = _msgSender();
    _spendAllowance(from, spender, amount);
    _transfer(from, to, amount);
    return true;
}
```

TransferFrom: This function enables an address to receive tokens from another address that has granted permission for the transfer.

ERC-20 Token Standard

```
/**
 * @dev updates `owner`'s allowance for `spender` based on spent `amount`
 *
 * Might emit an `approval` event.
 *
 */
function _spendAllowance(address owner, address spender, uint256 amount) internal virtual {
    uint256 currentAllowance = allowance(owner, spender);
    if (currentAllowance != type(uint256).max){
        require(currentAllowance >= amount, „ERC20: insufficient allowance“);
        unchecked{
            _approve (owner, spender, currentAllowance – amount);
        }
    }
}
```

ERC-20 Token Standard

```
/**
 * @dev Atomically increases the allowance granted to spender by the caller
 *
 * Emits a `Approval` event.
 *
 * Requirements:
 *
 * - `spender` cannot be the zero address.
 */
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool){
    address owner = _msgSender();
    _approve(owner, spender, allowance (owner, spender) + addedValue);
    return true;
}
```

ERC-20 Token Standard

```
/**
 * @dev Atomically decreases the allowance granted to spender by the caller
 *
 * Emits a `Approval` event.
 *
 * Requirements:
 *
 * - `spender` cannot be the zero address.
 * - `spender` must have allowance for the caller of at least.
 * - `subtractedValue`.
 */
function decreaseAllowance(address spender , uint256 subtractedValue) public virtual returns (bool){
    address owner = _msgSender();
    uint256 currentAllowance = allowance (owner, spender);
    require(currentAllowance >= subtractedValue, „ERC20: decreased allowance below zero“);
    unchecked {
        _approve (owner, spender, currentAllowance – subtractedValue);
    }

    return true;
}
```

ERC-20 Token Standard

```
/**
 * @dev creates `amount` tokens and assigns them to `account`
 *
 * Emits a `Transfer` event with `from` set to the zero address .
 *
 * Requirements:
 *
 * - `account` cannot be the zero address.
 */
function _mint(address account, uint256 amount) internal virtual {
    require(account != address(0), "ERC20: mint to the zero address");

    _beforeTokenTransfer (address(0), account, amount);

    _totalSupply += amount;
    _balances[account] += amount;
    emit Transfer( address(0), account, amount);

    _afterTokenTransfer(address(0), account, amount);
}
```

ERC-20 Token Standard

```
/**
 * @dev Destroys `amount` tokens from `account` reducing the total supply.
 *
 * Emits a `Transfer` event.
 *
 * Requirements:
 *
 * - `account` cannot be the zero address.
 * - `account` must have at least `amount` tokens .
 */
function _burn(address account, uint256 amount) internal virtual {
    require(account != address(0), "ERC20: burn from the zero address");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require (accountBalance >= amount, „ERC20: burn amount exceeds balace“);
    unchecked{
        _balances[account] = accountBalance - amount ;
    }
    _totalSupply -= amount;
    emit Transfer(account, address(0), amount);
    _afterTokenTransfer(account, address(0), amount);
}
```

ERC-721 Token Standard

ERC20 tokens are not suitable to represent ownership of individual and unique assets like a house or a unique artwork. Therefore, the ERC721 token standard was created in January 2018 as a standard interface for non-fungible tokens (NFT). Each ERC721 token is distinguishable by a unique ID.

Use cases

- Digital goods
 - In-Game items
 - Collectables
 - Music
 - etc.
- Physical property
 - Real estate
 - etc.
- Negative value assets
 - Loans
 - etc.



ERC-721 Token Standard

Interface

```
pragma solidity ^0.4.20;

/// @title ERC-721 Non-Fungible Token Standard
/// @dev See https://eips.ethereum.org/EIPS/eip-721

interface ERC721 {
    function balanceOf(address _owner) external view returns (uint256);
    function ownerOf(uint256 _tokenId) external view returns (address);
    function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes data) external payable;
    function safeTransferFrom(address _from, address _to, uint256 _tokenId) external payable;
    function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
    function approve(address _approved, uint256 _tokenId) external payable;
    function setApprovalForAll(address _operator, bool _approved) external;
    function getApproved(uint256 _tokenId) external view returns (address);
    function isApprovedForAll(address _owner, address _operator) external view returns (bool);

    event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId);
    event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId);
    event ApprovalForAll(address indexed _owner, address indexed _operator, bool _approved);
}
```

Design Patterns

- **Programming idioms** are language-specific patterns for recurring programming problems.

```
// Java idiom for generating random number.  
Random rand = new Random();  
int diceRoll() {  
    return rand.nextInt(6) + 1;  
}
```

- Idioms are on a lower abstraction level than **design patterns** which are template solutions for recurring software engineering problems.
- *OpenZeppelin* is a library that automates operations and delivers reusable, secure, tested and community-audited code. Most of the critical building blocks that are needed for a contract are already pre-programmed in it, so users should utilize the existing library instead of writing their own code. In this section, we will go through the most prevalent idioms of Solidity smart contract programming, using the OpenZeppelin library.
- *SafeMath*¹ is library that validates if an arithmetic operation would cause an integer overflow/underflow.
 - Until Solidity *version 0.8*, it had to be manually included and utilized by the smart contract developer
 - Since *version 0.8*, it is implemented on the language level

A **Solidity idiom** is a practice-proven code **pattern** for a **recurring coding problem**.

Design Patterns: Access Restriction

Description

- Each contract deployed on the main network is publicly accessible.
- Since all external and public functions can be called by anyone, third parties might execute a function on a contract they should not be allowed to.
- Misconfigured access restrictions led to the largest Ethereum thefts so far¹.

Participants

- An entity that calls a publicly accessible function in a smart contract.
- A smart contract which is called by a transaction or a message.

Applicability

- To protect contract functions from unauthorized calls.
- Examples for such functions: `selfdestruct()`, `mint()`

Design Patterns: Access Restriction

Solution

Restrict access of other accounts to execute functions and to modify the state of a contract. In Solidity, access restriction can be achieved by implementing proper **function modifiers** that check if the caller is allowed to execute the actual function logic. To make the contract code more maintainable, the authorization logic is usually put in a separate contract.

Ownable Contract

```
contract Ownable {  
    address public owner;  
  
    constructor() public {  
        owner = msg.sender;  
    }  
  
    /**  
     * @dev Throws if called by any account other than the owner.  
     */  
    modifier onlyOwner() {  
        require(msg.sender == owner);  
        _;  
    }  
}
```

Contract which implements Ownable

```
/**  
 * @dev Allows the current owner to transfer control  
 * @param newOwner The address to transfer ownership to.  
 */  
function transferOwnership(address newOwner) public onlyOwner {  
    require(newOwner != address(0));  
    owner = newOwner;  
}  
  
/**  
 * @dev Allows the current owner to relinquish control */  
function renounceOwnership() public onlyOwner {  
    owner = address(0);  
}  
}
```

Design Patterns: Oracles

Problem Description

Smart contracts can't access any **data** from **outside** the **blockchain** on their own. There are no HTTP or similar network methods implemented to call or access external services. This is on purpose to **prevent non-deterministic behavior** once a function is called (e.g., there are also no functions to generate random values).

Participants

- A smart contract which **requires data** from external sources
- An external party that is willing to **provide data** from external sources via a **separate smart contract**

Applicability

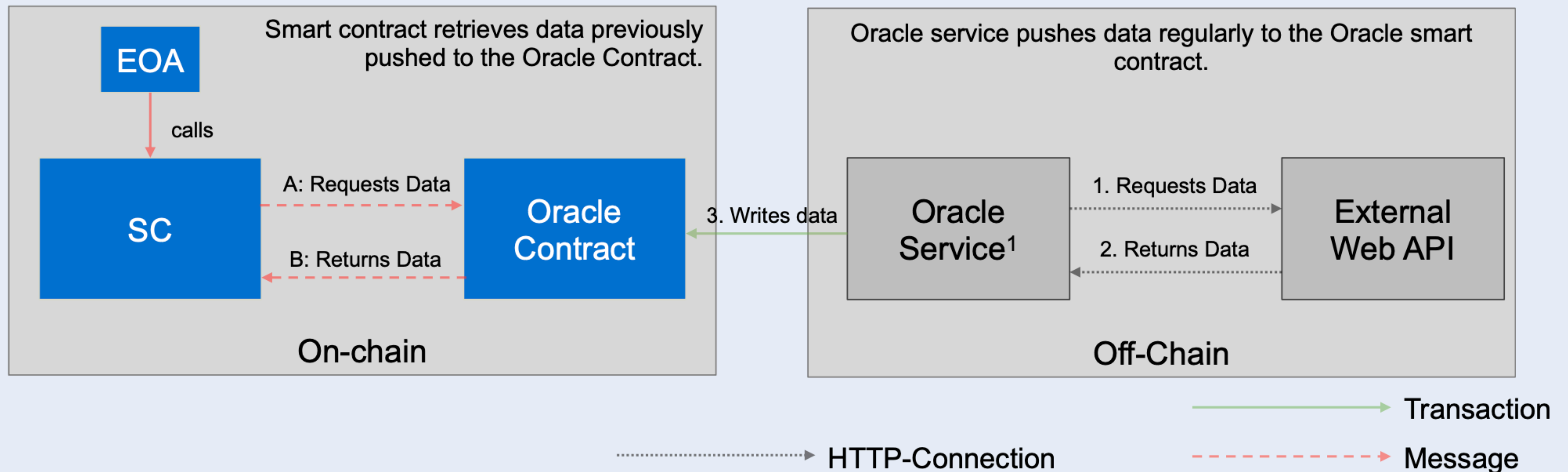
Any scenario in which a smart contract **relies on external data for computation** of future states.

Design Patterns: Synchronous Oracles

Solution

Currently, the **only way** to write **external data** (e.g. weather data, traffic data etc.) to smart contracts is to **use oracles**. Oracles are basically third-party services that collect data from web services and write the data via a specially designed smart contract to the blockchain. Other smart contracts can call the oracle contract to get the data. We differentiate between **synchronous** and **asynchronous** oracles.

Synchronous Oracle



Design Patterns: Synchronous Oracles

A simple Oracle contract for the current market prices of Ethereum and Bitcoin.

```
contract SimpleOracle {  
    address public controller;  
    enum Coins {Ethereum, Bitcoin}  
    Coins public coins;  
    mapping (uint => uint) public prices;  
  
    constructor() public {  
        controller = msg.sender;  
    }  
  
    modifier isController {  
        require(msg.sender == controller);  
        _;  
    }  
  
    function update(uint coin, uint price) public isController {  
        if(coin != uint(coins.Ethereum) && coin != uint(coins.Bitcoin)) {  
            revert();  
        }  
        prices[coin] = price;  
    }  
  
    function getPrice(uint coin) public view returns(uint price) {  
        return prices[coin];  
    }  
}
```

The controller of the Oracle contract

The storage that represents the market data

Controller is set by contract deployment

An update function that lets the controller of the contract set the current market prices

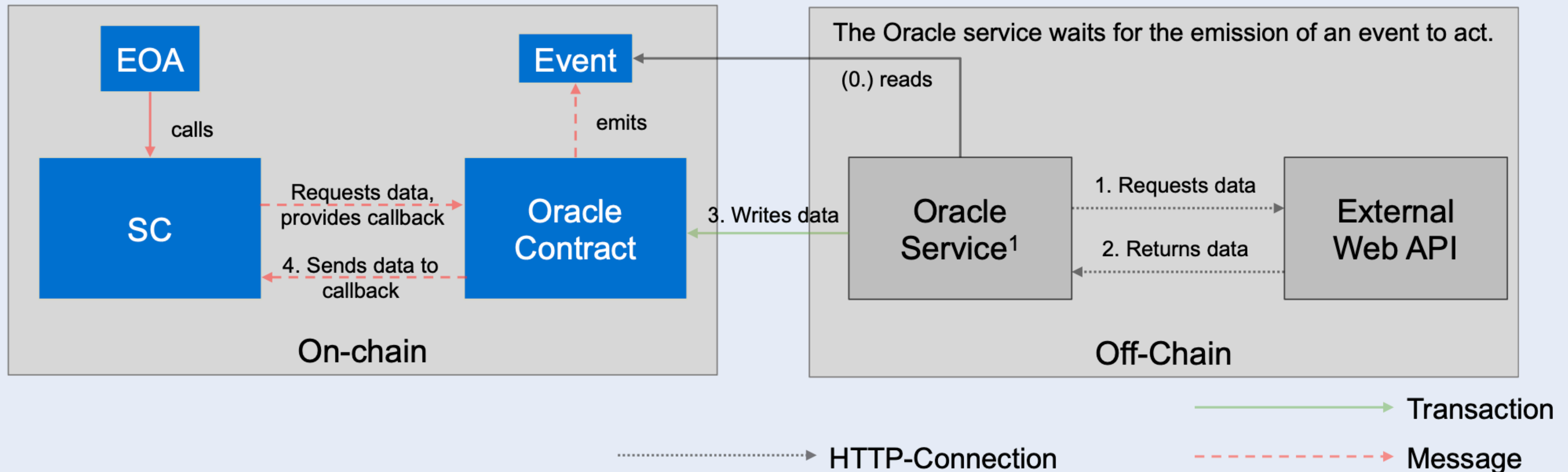
A publicly accessible view function that returns the current price of a particular coin

Design Patterns: Asynchronous Oracles

Solution

In contrast to the synchronous oracle (in which the data is periodically pushed to the oracle contract and retrieved by the smart contract), the **asynchronous oracle** acts as a proxy for the oracle service to send fresh data to the original smart contract.

Asynchronous Oracle



Design Patterns: Asynchronous Oracles

```
pragma solidity ^0.4.26;
```

```
contract Client {  
  address public oracle;  
  uint public a;  
  
  constructor(address _oracle) public {  
    oracle = _oracle;  
  }  
}
```

1. execute

```
function getOracleData() public {  
  Oracle oraclecontract = Oracle(oracle);  
  oraclecontract.invokeOracle();  
}
```

```
function __OracleCallback(uint _a) external onlyOracle {  
  a = _a;  
}
```

4. write back

```
modifier onlyOracle() {  
  require(msg.sender == oracle, "not oracle");  
  _;  
}
```

```
contract Oracle {  
  address public owner;  
  
  constructor () public {  
    owner = msg.sender;  
  }  
}
```

```
event OracleInvoked(address sender);
```

```
function invokeOracle() public {  
  emit OracleInvoked(msg.sender);  
}
```

```
function callBack(uint _a, address _client) public onlyOwner {  
  Client clientcontract = Client(_client);  
  clientcontract.__OracleCallback(_a);  
}
```

```
modifier onlyOwner() {  
  require(msg.sender == owner, "not owner");  
  _;  
}
```

```
}
```

2. reads

External
Oracle Server

3. Invokes
(issues a tx)

Design Patterns: Oracles

Advantages

- Enables data retrieval from external sources
- Either easy to use (Synchronous Oracle) or fresh data (Asynchronous Oracle)
- Can be used for varying purposes (randomness, stock data, weather data, ...)

Disadvantages

- Costly in terms of gas consumption
- Dependence on a third party
 - In terms of data manipulation (Oracle owner can manipulate data)
 - In terms of availability (Oracle service could be offline → Smart contract not functioning)

Design Patterns: Randomness

Problem Description

Solidity **does not provide any functions** that **generate random numbers**. This is due to the forced **deterministic** behavior of the EVM. Having random numbers would make it impossible for other nodes to validate the correct output of a function.

Participants

- A smart contract which **requires random numbers**
- (Potential participants which provide the random number)

Applicability

Any scenario in which a smart contract **relies on random numbers for computation** of future states.

Design Patterns: Randomness

Solution

Oracles are usually used to access data from outside of the blockchain. Additionally, oracles can provide random numbers, either from local sources or from services (e.g., WolframAlpha).

Limitations

- Costly in terms of gas consumption
- Dependence on a third party in terms of data manipulation (Oracle owner can manipulate data)
- Potential predictability

Design Patterns: Randomness

Solution

A commonly used approach is to create pseudo-random numbers and use the hash of the currently proposed block. The hash of the block is not known upfront and therefore more or less random. In theory, a block proposer could affect and manipulate the block hash by deciding which transactions are included in the block and which are not.

Example Code

```
contract SimpleRandom {  
    function randomNumber() public view returns (uint) {  
        return uint(blockhash(block.number - 1));  
        // -1 for the latest block where the transaction with the function call is included  
    }  
}
```

Limitations

- Block proposer could try to delay transactions that do not lead to the desired result
- Only relevant for small amounts of money (< mining reward)
- Potential predictability by miner

Design Patterns: Randomness

Solution

Another potential approach is the commit and reveal scheme, which utilizes the hiding property of cryptographic hash functions. It consists of two phases:

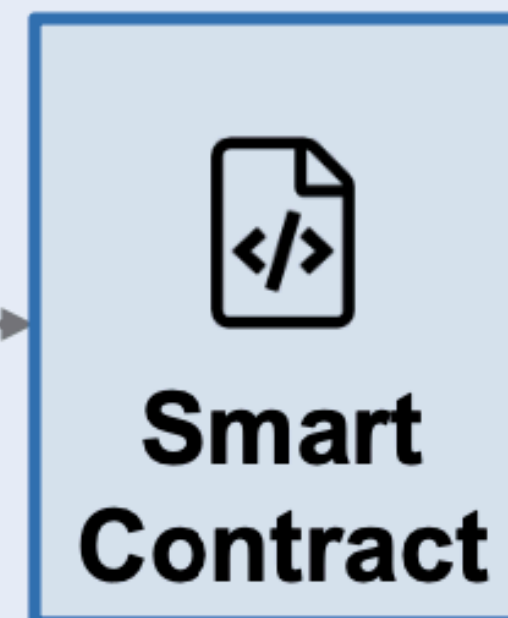
1. Users (>1) hash their random number r , hide it by concatenating with x , and publish it to the smart contract.
2. Users reveal their random number and the smart contract calculates a random number from these commits.

Phase 1: Commit

Secret x_a
Random
Number r_a



commit



commit



Secret x_b
Random
Number r_b



Design Patterns: Randomness

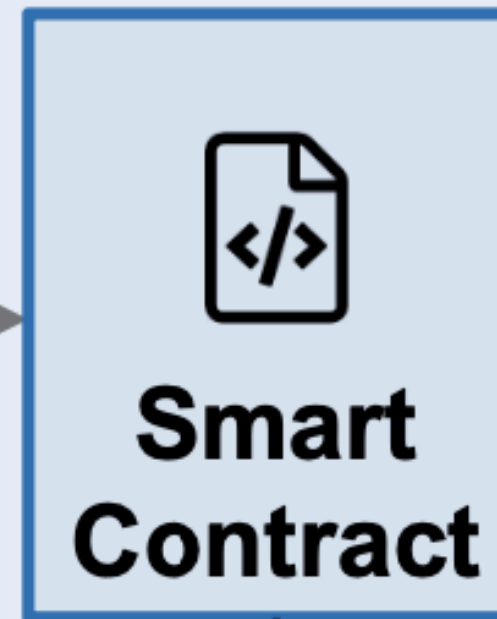
Phase 2: Reveal

Secret x_a
Random
Number r_a



reveal(r_a , x_a)

transaction



transaction

reveal(r_b , x_b)



Secret x_b
Random
Number r_b

Computes
 $(r_a + r_b) \% 2$

$(r_a + r_b) \% n$ could return n different results.
If $\text{player} == n$, then you can select a winner
with the provided random numbers.

Limitations

- Requires user interaction
- Users could withhold their reveals
 - Possible solution: If information is not revealed, the stakes are distributed among other players

Design Patterns: Versioning

The code of deployed contracts is immutable by design which makes evolving them difficult.

However, sometimes it is necessary to replace a contract with a newer version, e.g., to fix a bug or add new functionality.

Replacing a contract with another version is challenging:

- Each contract deployed on the blockchain gets a new address
- Entities referencing a previous version of a contract need to be notified about the update
- Each contract has its own state
- State migration is not straightforward since the more recent contract must inherit not only the data but all permissions as well
- Potential state inconsistencies when the previous contract version is still used by some entities

Design Patterns: Data Segregation

Problem Description

- Each smart contract maintains its own state.
- Most smart contracts do **not separate state data** from **business logic**.
- **Close coupling** becomes a **problem** whenever the contract needs to be **replaced by a newer version**.
- A contract update requires a migration process for its state.

Participants

- An entity that wants to replace a smart contract with a more recent version.
- A smart contract that implements some business logic.

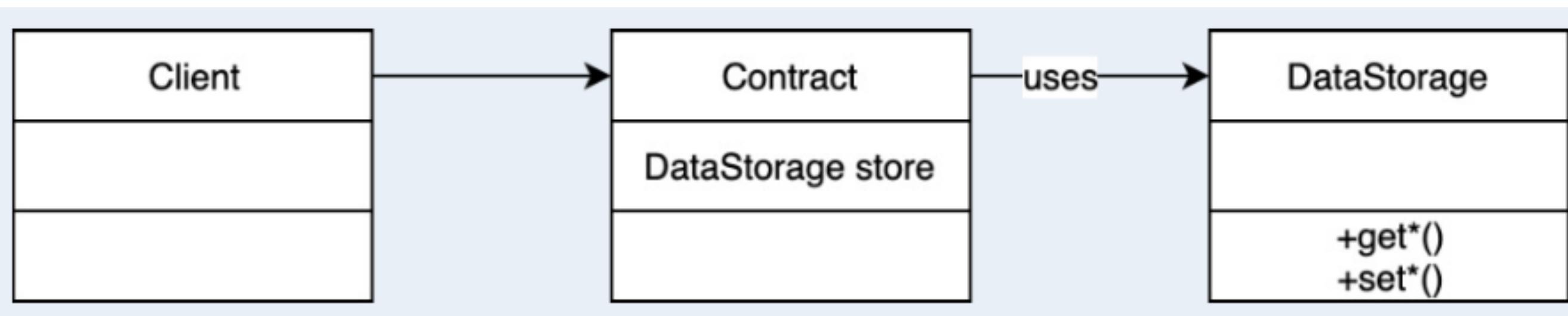
Applicability

- Scenarios where a vulnerability is found in a smart contract.
- Whenever business logic needs to be updated.

Design Patterns: Data Segregation

Solution

- **Decoupling the business logic** from the **state data** by using two separate contracts.
- One contract implements all the functions for the business logic.
- Another one acts as a **raw data storage contract**.
- The storage contract must implement an interface that provides all the necessary getter and setter functions.
- Whenever the business logic needs to be updated, a new contract could access the data through the storage contract.



```
contract DataStorage {
    // Whatever types the contract needs to store...
    mapping(bytes32 => uint) uintStore;

    function getIntValue(bytes32 key) public view returns (uint) {
        return uintStore[key];
    }

    function setIntValue(bytes32 key, uint value) public {
        uintStore[key] = value;
    }
}
```

Design Patterns: Data Segregation

Advantages

- Separation of concerns – Low coupling
- Updating the business logic of a contract is straight forward
- Migration to a new business logic does not require separate migration of the state

Disadvantages

- The user has to trust the maintainer of the contract
- One has to maintain multiple contracts at once
- Introduces overhead to ensure access rights and permissions
- In case of an update, applications need to be notified about the contract address of the new version
- Requires proper lifecycle management

Design Patterns: Proxy

Problem Description

- **Replacing a smart contract** with a newer version implies that the newly deployed contract has a **different account address** than the older one.¹
- This behaviour leads to the problem that all entities using the older contract version **must be notified** about the **new contract address**.
- All contracts that reference an old version need to update their state.

Participants

- An entity that wants to replace a smart contract with a more recent version.
- A smart contract that implements some business logic.

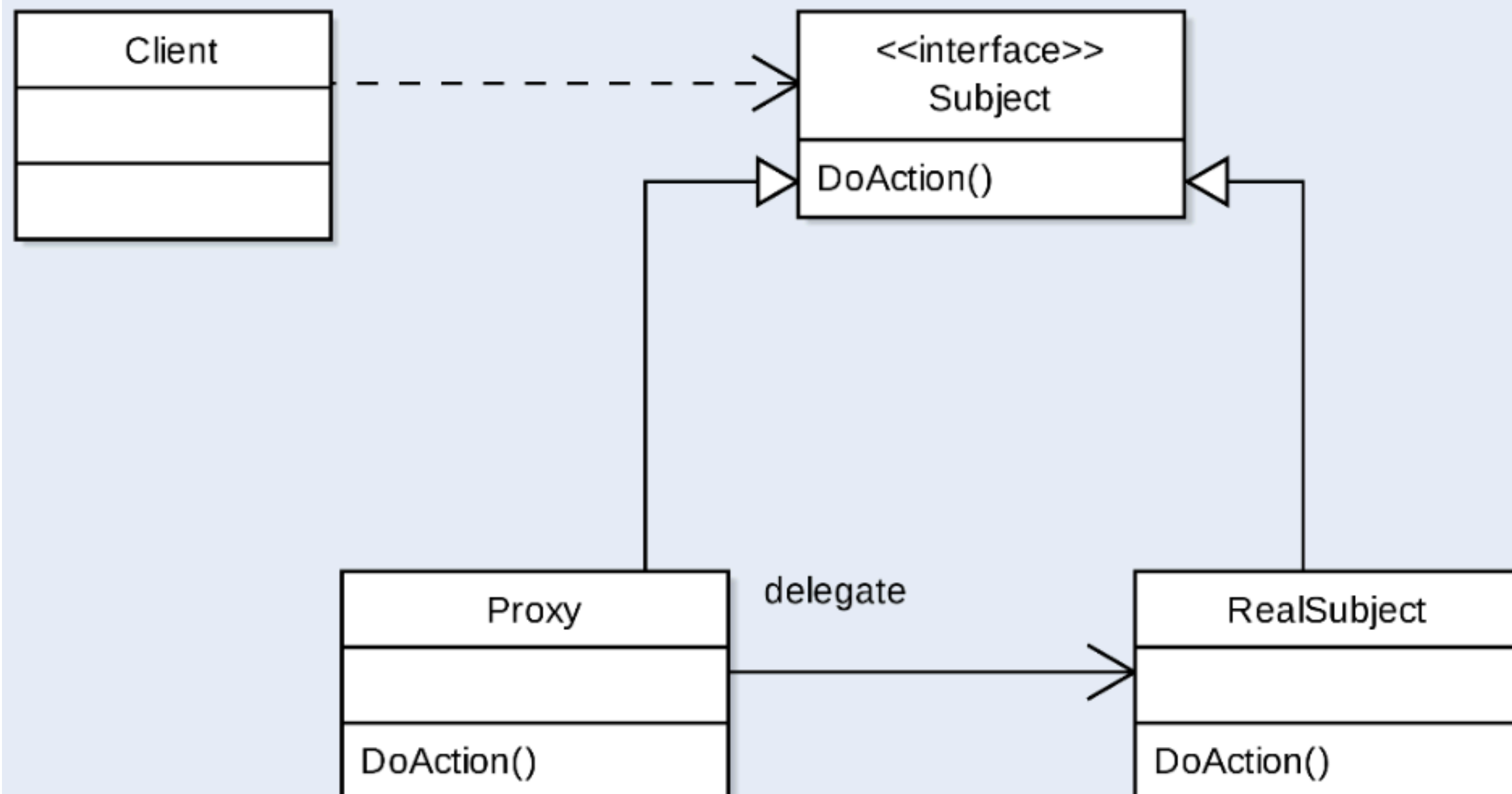
Applicability

- Scenarios where a vulnerability is found in a smart contract,
- Whenever the business logic needs to be updated in a way that the address of the contract does not change.

Design Patterns: Proxy

Solution

- Separate the business logic and the data from the actual, application-facing contract.
- Implement a proxy contract which forwards all calls to the contract with the business logic.



```
import "../authorization/Ownership.sol";
import "../Proxy.sol";
contract Proxy is Owned {
    address logicAddress;

    function doSomething() public onlyOwner {
        Logic l = Logic(logicAddress);
        l.doSomething();
    }

    function updateLogicAddress(address _address) public onlyOwner {
        logicAddress = _address;
    }
}
```

Design Patterns: Proxy

Advantages

- Separation of concerns – Low coupling
- Updating the business logic of a contract is straightforward
- Deployments of new contract versions do not require an address change
- Bugs in contracts can be fixed

Disadvantages

- One has to trust that the maintainer of the proxy to not replace the business logic with malicious code
- The interface is still immutable, it is not possible to add new callable functions (at least without using a hack through the fallback function and delegatecall()-method)
- Proxies make it harder to read and verify the logic of a smart contract
- Introduces overhead to ensure access rights and permissions
- One has to maintain multiple contracts at once

Design Patterns: Tight Variables

Description

- Deploying a smart contract costs gas depending on the complexity of the code.
- Tight variable packing reduces the amount of gas utilized when deploying and interacting with a contract. The tight packing idiom is simple to use and has no effect on the contract logic.

Participants

- Deployer of a smart contract.
- The modifications only affect how data is kept, no other entities dealing with the contract are affected.

Applicability

- Anytime a smart contract is going to be deployed.
- Using the lowest data type feasible while yet ensuring accurate code execution.
- Group all data types that should belong together into a single 32-byte slot and declare them one by one in the code. Grouping is critical as the EVM stores variables in the order they are provided.

Design Patterns: Tight Variables

Example:

In this example, there are two structs where one makes use of the tight variable packing idiom and the other does not. The one that utilizes packing can pack **a** and **c** into the same slot since they do not exceed 32 bytes when combined. Thus, 2 storage slots in total are sufficient to store the struct. Since **b** interrupts the order in the other struct, it requires 3 storage slots.

```
contract StructWithPacking{  
    // 2 storage slots  
    struct CheapStruct {  
        uint128 a; // 16 bytes  
        uint128 c; // 16 bytes  
        uint256 b; // 32 bytes  
    }  
}
```

```
contract StructWithoutPacking{  
    // 3 storage slots  
    struct ExpensiveStruct {  
        uint128 a; // 16 bytes  
        uint256 b; // 32 bytes  
        uint128 c; // 16 bytes  
    }  
}
```